<SAFE BANK|

Relational Model>

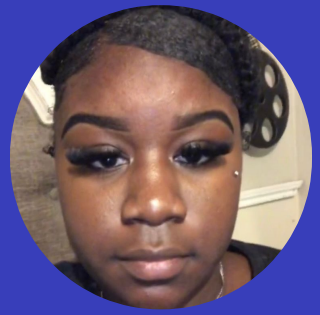
START >



TEAM



ALIRAT OLADEJO



MARIA BINGHAM



AMI EGUIA



ABRIL REYES



JOY
OLOWOGBADE

< 00 > Goals

< 01 > DDL Script

< 02 > Data Integrity

< 03 > DML Script

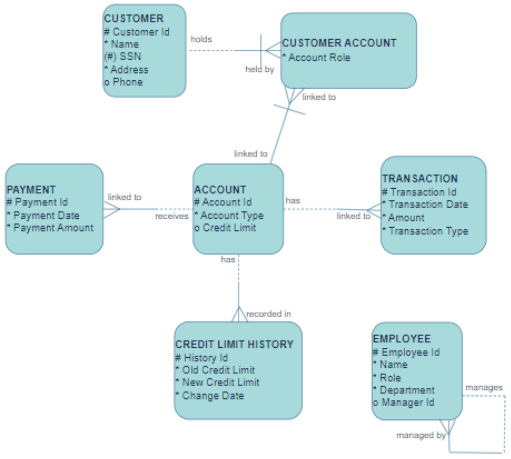
< 04 > Query & Test

< 05 > Reflection



<Goals>

CONVERT OUR ERD LOGICAL MODEL INTO A RELATIONAL DATABASE

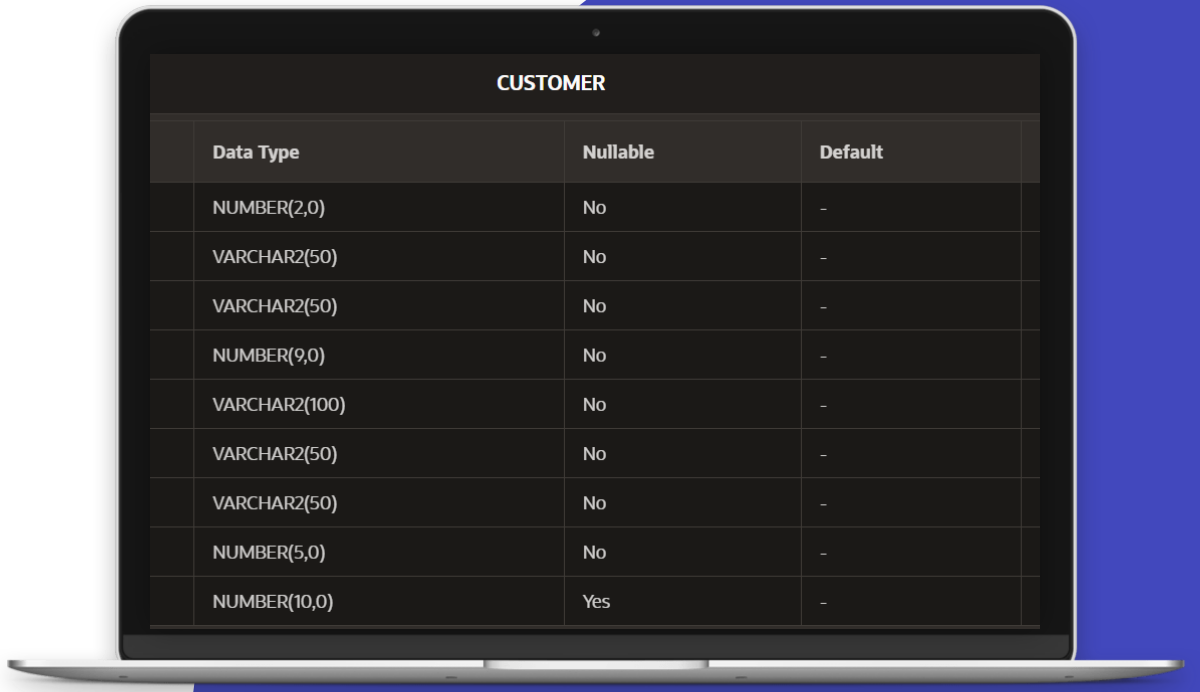




<01>
STEP 1

DDL Script Creation

START >



// DDL SCRIPT CREATION

We've created multiple tables based off our previous ERD model of our credit card management system using Data Definition Language in Oracle Apex.

```
STEP 1 & 2: DDL SCRIPT CREATION & ENFORCE DATA INTEGRITY
-- Create Customer table
CREATE TABLE Customer (
    CustomerId NUMBER(2) PRIMARY KEY,
    FirstName VARCHAR2(50) NOT NULL,
    LastName VARCHAR2(50) NOT NULL,
    SSN NUMBER(9) UNIQUE NOT NULL,
    Street VARCHAR2(100) NOT NULL,
    City VARCHAR2(50) NOT NULL,
    State VARCHAR2(50) NOT NULL,
    ZipCode NUMBER(5) NOT NULL,
    Phone NUMBER(10)
);
```

<02>
STEP 2

Enforce Data Integrity

START >



// CONSTRAINTS

We enforced data integrity by adding multiple contraints to each table. We also inserted data to check for errors. If we had an error, we would have to recheck our data according to the contrainsts.



```
CREATE TABLE Account (

AccountId NUMBER(3) PRIMARY KEY,

AccountType VARCHAR2(10) NOT NULL CHECK (AccountType IN ('Checking', 'Savings', 'Credit')),
CreditLimit DECIMAL(10, 2)

);
```

// EXAMPLE

Above is an example of a table creation and its constraints.

<03>
STEP 3

Data Population & Manipulation

START >



// DATA POPULATION

We've created 7 fake data instances to insert into our tables for data population.

CUSTOMER

EDIT	CUSTOMERID	FIRSTNAME	LASTNAME	SSN	STREET	CITY	STATE	ZIPCODE	PHONE
	2	Jane	Smith	234567890	456 Oak St	Dallas	TX	75201	2345678901
	5	Eve	Davis	567890123	654 Cedar St	El Paso	TX	79901	5678901234
	1	John	Doe	123456789	123 Elm St	Austin	TX	78701	1234567890
	3	Alice	Johnson	345678901	789 Pine St	Houston	TX	77001	3456789012
	6	Charlie	Wilson	678901234	987 Maple St	Fort Worth	TX	76101	6789012345
	4	Bob	Brown	456789012	321 Birch St	San Antonio	TX	78201	4567890123

// DML (DATA MANIPULATION LANGUAGE)

```
1  select * from account order by accountid asc;
```

Results | Explain | Describe | Saved SQL | History

ACCOUNTID	ACCOUNTTYPE	CREDITLIMIT
101	Checking	5000
102	Savings	7500
103	Credit	1500
104	Checking	4000

We've used DML to manipulate our tables in many different ways. Here are some examples.

```
1  update account set creditlimit=5500.00 where accountid=101;
2  select * from account order by accountid asc;
```

Results | Explain | Describe | Saved SQL | History

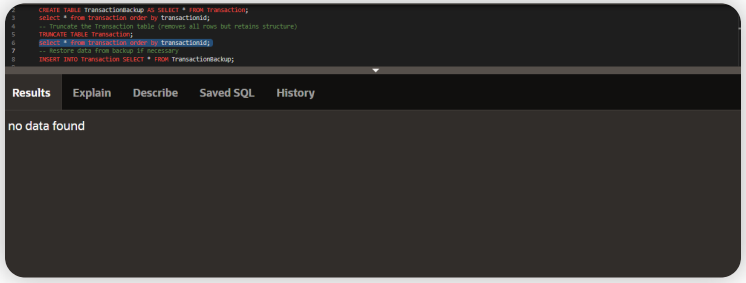
ACCOUNTID	ACCOUNTTYPE	CREDITLIMIT
101	Checking	5500
102	Savings	7500
103	Credit	1500
104	Checking	4000



```
select * from account order by accountid asc;
-- Add a Lastupdated column to account
ALTER TABLE ACCOUNT
ADD Lastupdated DATE;
-- Update the Lastupdated column
UPDATE Account
SET Lastupdated = SYSDATE;
```

Results | Explain | Describe | Saved SQL | History

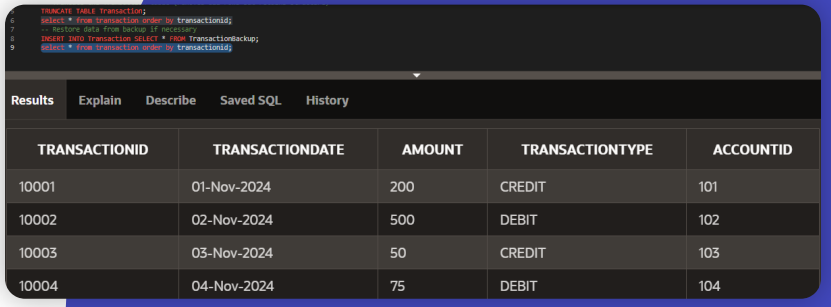
ACCOUNTID	ACCOUNTTYPE	CREDITLIMIT	LASTUPDATED
101	Checking	5500	03-Dec-2024
102	Savings	7500	03-Dec-2024
103	Credit	1500	03-Dec-2024
104	Checking	4000	03-Dec-2024



```
CREATE TABLE TransactionBackup AS SELECT * FROM Transaction;
select * from transaction order by transactionid;
-- Truncate the Transaction table (removes all rows but retains structure)
TRUNCATE TABLE Transaction;
select * from transaction order by transactionid;
-- Restore data from backup if necessary
INSERT INTO Transaction SELECT * FROM TransactionBackup;
```

Results | Explain | Describe | Saved SQL | History

no data found



```
TRUNCATE TABLE Transaction;
select * from transaction order by transactionid;
-- Restore data from backup if necessary
INSERT INTO Transaction SELECT * FROM TransactionBackup;
select * from transaction order by transactionid;
```

Results | Explain | Describe | Saved SQL | History

TRANSACTIONID	TRANSACTIONDATE	AMOUNT	TRANSACTIONTYPE	ACCOUNTID
10001	01-Nov-2024	200	CREDIT	101
10002	02-Nov-2024	500	DEBIT	102
10003	03-Nov-2024	50	CREDIT	103
10004	04-Nov-2024	75	DEBIT	104

<04>
STEP 4

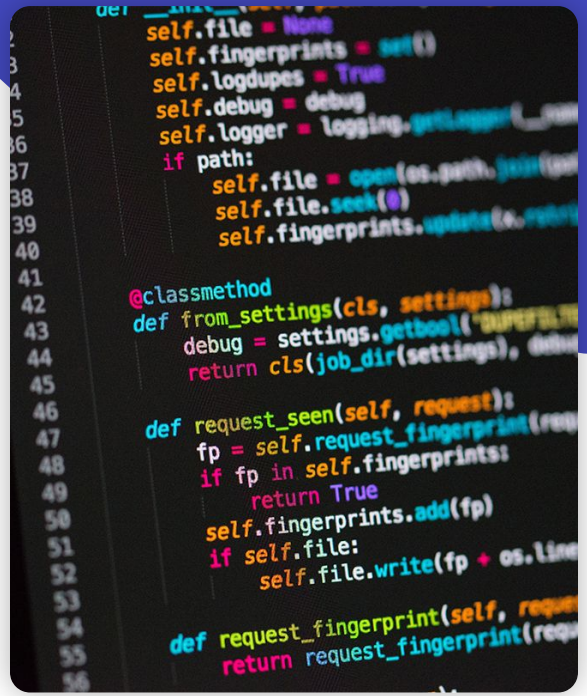
Querying & Testing the Database

START >



// QUERYING & TESTING THE DATABASE

We've used many different ways to query and test our database. We've used select, order by, join, others.



```sql
-- List all accounts held by a customer (e.g., CustomerId = 1)
SELECT C.FirstName, C.LastName, A.AccountType, A.CreditLimit
FROM Customer C
JOIN CustomerAccount CA ON C.CustomerId = CA.CustomerId
JOIN Account A ON CA.AccountId = A.AccountId
WHERE C.CustomerId = 1;
```

Results | Explain | Describe | Saved SQL | History

FIRSTNAME	LASTNAME	ACCOUNTTYPE	CREDITLIMIT
John	Doe	Checking	5500

<05>
STEP 5

Documentation & Reflection

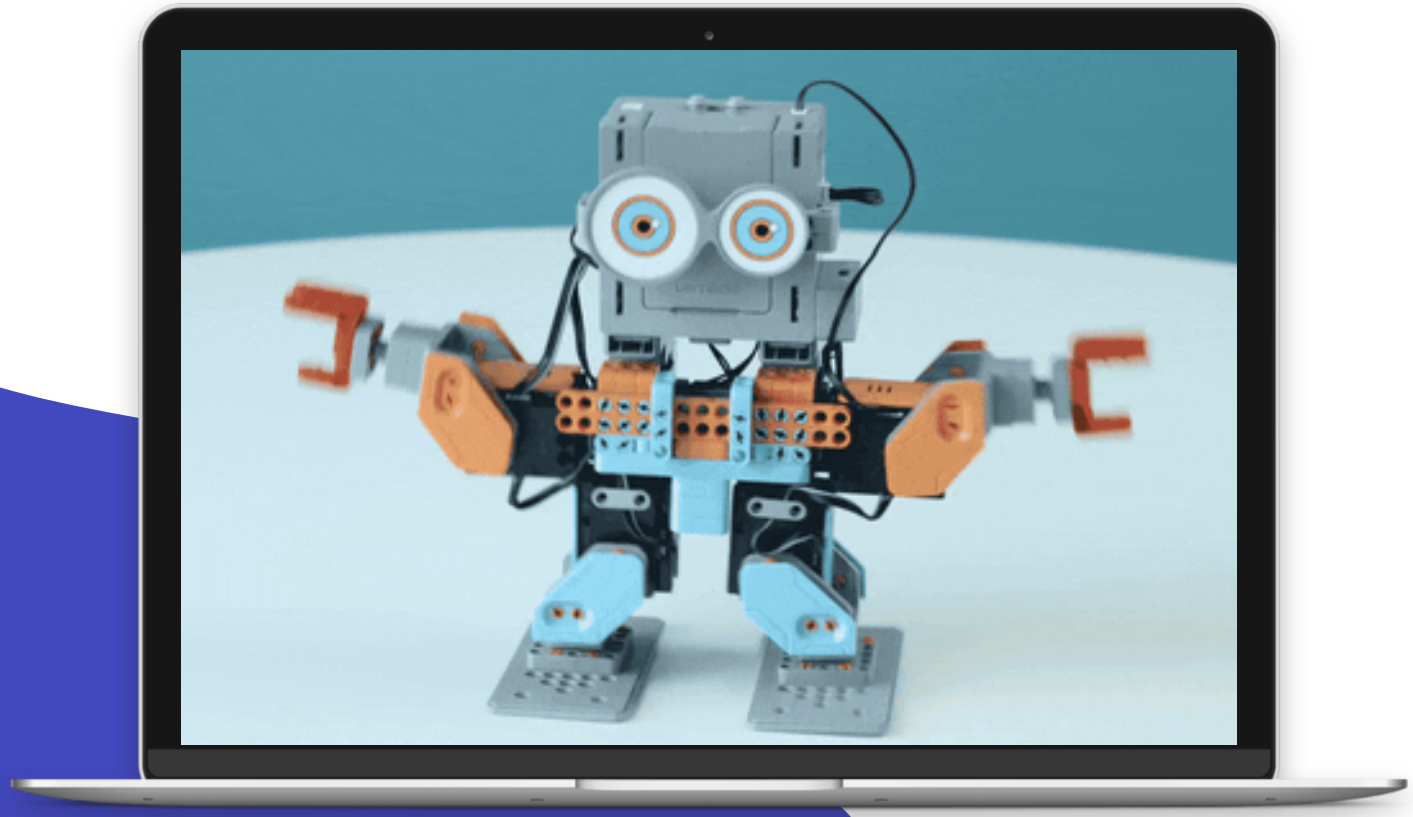
START >



REFLECTION

This project taught us a lot about turning a logical ERD into a working database. Adding things like foreign keys and CHECK rules showed me how important it is to keep data accurate and consistent. Figuring out tricky parts, like how to handle joint accounts and employee relationships, was challenging but made us better at solving problems. Overall, this project showed us how important it is to plan carefully, test often, and explain my work clearly to create a successful database.

¡THANK YOU!



Thank You |

